

Chapitre V : Les réseaux de neurones artificiels RNA

1. Introduction

Ce chapitre commence par une introduction dans laquelle elle décrit l'histoire de l'évolution des réseaux de neurones artificiels RNA. Cette partie est suivie par la présentation du neurone artificiel ces formulations mathématiques et ses caractéristiques de base. La section suivante de ce chapitre se concentre sur la présentation des différentes architectures de base des RNAs décrites par le perceptron à multi couche PMC et les réseaux de neurones convolutifs CNN. Dans cette partie, les différentes caractéristiques paramétriques et écritures algébriques sont développées.

2. Historique

Les premières tentatives de la modélisation des réseaux neuronaux ont démarré en 1943 par [Warren McCulloch](#) et [Walter Pitts](#). Ils ont instauré les bases théoriques de fonctionnement de base de ces systèmes inspirés du fonctionnement du cerveau humain. En effet, **Ils considèrent que le fonctionnement des neurones naturels est l'équivalent de celui des portes logiques** en mathématique. Mais ils ne donnent aucune indication sur l'adaptation des coefficients synaptiques.

En 1949, Donald Hebb présente dans son ouvrage qui définit une règle d'adaptation. Cette règle introduit des coefficients synaptiques, elle permet de modifier leurs valeurs en fonction de l'activité des unités qu'ils relient. Cette règle connue sous le nom de la règle de Hebb est presque utilisée par la plupart des modèles actuels.

En 1958, Frank Rosenblatt développe le modèle primaire du Perceptron. C'est un réseau de neurones inspiré du système visuel. Il possède deux couches de neurones : une couche de perception et une couche liée à la prise de décision. Une vague d'enthousiasme est provoquée et annonce que le perceptron pourra bientôt faire de l'exploration spatiale, de prendre des décisions et de traduire des langues.

L'année 1969 est importante et cruciale pour les RNs. En effet, Marvin Lee Minsky et Seymour Papert ont publié une critique des propriétés du Perceptron dans laquelle ils estiment qu'imiter le cerveau est trop complexe mettant en évidence certaines limitations théoriques du perceptron, ces derniers sont considérés généralement comme des classificateurs linéaires et ne peuvent pas traiter des problèmes non linéaires. Ces limitations ont contribué à mettre les modèles des RNA dans une impasse. C'est ainsi que la recherche sur les RNs perd, petit à petit, l'essentiel de ses financements publics et industriels et les fonds destinés à l'intelligence artificielle furent redirigés plutôt vers la logique formelle et les systèmes experts.

Toutefois, plusieurs tentatives de recherches se sont concentrées sur ces limitations et ont contribué à l'intégration des fonctions non linéaires dans les neurones, de l'algorithme du gradient descendant dans l'ajustement des coefficients synaptiques. C'est à partir de l'année l'application de l'algorithme de la rétro propagation du gradient descendant au RNA par David Rumelhart, Ronald J. Williams et Geoffrey Hinton a provoqué sa révolution et son explosion vers divers études et applications. Son application est popularisée par le perceptron multi-couches, des réseaux récurrents, les réseaux convolutifs et par la suite des réseaux profond.

3. Le neurone formel

2.1. Le neurone biologique

Le neurone biologique est la cellule de base de notre cerveau. En effet, notre cerveau compte entre 86 et 100×10^9 neurones et pèse entre et Kg. L'activation des neurones est assuré par des flux de signaux pouvant atteindre Km/h. ces flux sont assurés dans des connexions entre les neurones qu'on appelle axones. Et on a approximativement km d'axones dans notre cerveau!!! D'une façon simplifiée un neurone est formé principalement par un corps cellulaire faisant le traitement des signaux des dendrites ramifiées, utilisées comme des antennes sensibles, des axones allongés qui assurent la communication et des synapses permettant la transmission des messages importants. Avec ces connexions biologiques de signaux électriques et chimiques, les neurones s'unissent pour construire nos pensées, nos sentiments et nos actions, créant ainsi un réseau d'une immense complexité. La figure ci-dessous représente le schéma simplifié d'un neurone biologique afin de faciliter la compréhension de son fonctionnement et passer par la suite à sa modélisation.

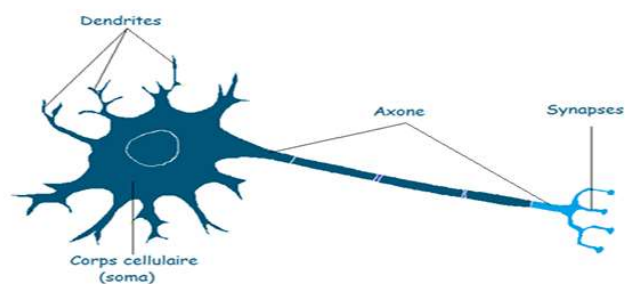


Figure 5.1. Représentation d'un neurone biologique

2.2. Le neurone Artificiel

Le neurone artificiel est une modélisation du neurone biologique. Il est caractérisé par ses entrées sa fonction d'activation (transfert) et sa sortie. Le traitement d'un neurone se fait en deux phases. La

première consiste au calcul de la
à appliquer à cette valeur une
figure ce dessous décrit ce traitement.

des ses entrées. La deuxième phase consiste
qui décrit l'activation du neurone. La

Figure 5.2. Présentation et notation des paramètres d'un neurone artificiel

Première phase :

$$s_j =$$

Avec :

x_i : l'ⁱème entrée,

$w_{j,i}$: Poids de la connexion entre l'ⁱème entrée et le j^{ème} neurone,

N_e : Nombre d'entrées,

b_j : Biais du j^{ème} neurone,

s_j : Activation du j^{ème} neurone

Deuxième phase :

Une fonction d'activation Φ non linéaire, calcule la sortie y_j en fonction de la valeur de la fonction d'activation, soit :

$$y_j =$$

Avec :

y_j : Sortie du j^{ème} neurone,

Φ : Fonction d'activation.

Dans le cas ou on utilise la fonction sigmoïde l'écriture devient :

$$y_j =$$

$$y_j =$$

2.3. Les fonctions d'activations

La fonction d'activation d'un neurone est l'équivalent du potentiel d'activation du neurone biologique. Cette fonction permet de déterminer le degré d'activation du neurone. Il existe plusieurs fonctions qui décrivent des comportements d'activation différents. Le tableau ci-dessous présente les principales fonctions d'activations utilisées.

Tableau 5.1. Principales fonctions de d'activation des neurones artificielles.

Fonction	Equation	Allure	Domaine d'activation	Type RNA

4. Architectures de base des RNAs

4.1 Les architectures non bouclées

Un RNA non bouclé se compose par un ensemble de neurones connectés entre eux. L'information circule dans une seule direction : de la couche d'entrée vers la couche de sortie sans retour en arrière. Ce principe s'appelle la propagation en avant de l'information (connu sous le nom : forward propagation). Le perceptron à multi couche est l'architecture la plus connu de ce type de RNA. Il est illustré dans la figure ci-dessous.

Figure 5.3. Architecture d'un RNA non bouclé à deux couches cachées et deux sorties : 3-3-4-2

4.2. Les architectures Bouclées

Les RNA Bouclés (appelé aussi réseaux récurrents RNN) se différent de l'architecture précédente par le type de la propagation de l'information. Cette propagation se fait selon le sens des connexions qui peuvent être des connexions de retour. Cette notion de retour introduit la notion de temps dans le traitement de la propagation de l'information. Les principales architectures sont connu sous le nom des reseau de neurone récurrents RNN et les réseaux LSTM (long short term memories) qui sont une variante avancé des RNN. La figure ci-dessous montre le principe d'un réseau de neurone bouclé.

Figure 5.4. Architecture d'un RNA bouclé

4.3 Les Principales architectures des RNA

Beaucoup d'architecture des RNA sont utilisés dans le domaine de l'apprentissage automatique. Avec l'intérêt croissant de leurs applications, l'évolution du développement des nouvelles architectures est toujours en croissance. Dans cette partie un intérêt particulier est accordé aux architectures : perceptron à multi couche PMC et les réseaux de neurones convolutifs CNN. Ces différentes architectures seront expliquer la partie suivante.

4.3.1 Les perceptrons à multi couches (PMC)

4.3.1.1. Présentation :

Les perceptrons multicouches (ou multi layers perceptron MLP) sont des types de réseau de neurones artificiels à **propagation avant**. Cette architecture est la base de la majorité des autres structures des RNA. C'est le modèle fondamental pour la construction de modèles des réseaux de neurones profond. Un PMC contient types de couches : une couche , une couche de et des couches . Chaque neurone fait le calcul de la **somme pondérée** et alimente une **fonction d'activation non linéaire** à sa sortie.

Ce type de réseau s'est imposé comme le réseau incontournable dans la classification binaire ou multi classes, dans la régression, dans la prédiction des série temporelles, dans les applications de traitement de langage naturel et précisément dans la construction des transformer. La figure ci-dessous représente l'architecture de base d'un PMC.

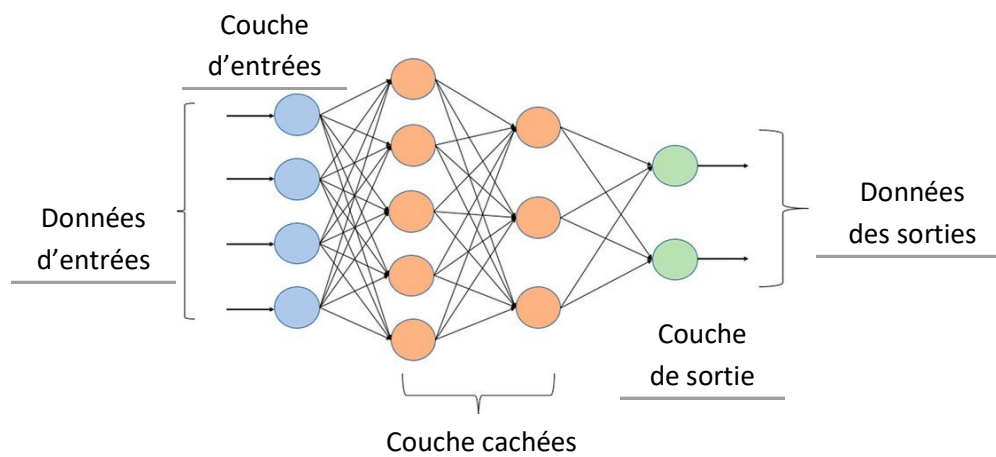


Figure 5.5. Architecture d'un PMC

Remarque

Si on note 'c'est le nombre de couches cachée.

est N_i et le nombre de neurone dans la couche

L'architecture d'un réseau PMC s'écrit :

Et le nombre de paramètres est calculé comme suit :

----- dans la 1^{ère} couche cachée

----- dans la 2^{ème} couche cachée etc.

Exemple

✓ Dans le cas de l'architecture 2-3-1, elle possède :

Le nombre de paramètres dans la couche cachée est : -----

Le nombre de paramètres dans la couche de sortie est : -----

On a donc un modèle à ----- paramètres.

✓ Dans le cas d'une architecture 5-10-3 :

Le nombre de paramètres dans la couche cachée est : -----

Le nombre de paramètres dans la couche de sortie est : -----

On aura en tous ----- paramètres.

✓ Dans le cas d'une architecture 2-3-5-1 :

Le nombre des paramètres dans la 1^{ère} couche cachée : -----

Le nombre des paramètres dans la 2^{ème} couche cachée : -----

Le nombre des de paramètres dans la couche de sortie -----

On aura en tous ----- paramètres.

4.3.1.2. Notation des paramètres d'un RNA type PMC

Soit un RNA de type PMC de structure : 4-5-3-2 décrit comme suit :

Exemple de notation

--	--	--	--

$$X = \begin{bmatrix} & \\ & \\ & \end{bmatrix} \quad \text{de dim}(X) = [\quad]$$
$$w^{[1]} = \begin{bmatrix} \\ \\ \\ \end{bmatrix} \quad \text{telque } \dim(w^{[1]}) = [\quad]$$

$$b^{[1]} = \begin{bmatrix} \\ \\ \\ \end{bmatrix} \quad \text{telque } \dim(b^{[1]}) = [\quad]$$

$$S^{[1]} =$$

$$\dim(w^{[1]}) = \quad ; \dim(X) = [\quad] \text{ et } \dim(b^{[1]}) = [\quad]$$

$$\text{donc } \dim(S^{[1]}) = \quad = [\quad]$$
$$a^{[1]} = \quad \dim(a^{[1]}) = [\quad]$$

La sortie pondérée de la deuxième couche cachée $S^{[2]}$ s'écrit alors

$$S^{[2]} =$$

$$\dim(w^{[2]}) = [\quad] ; \dim(a^{[1]}) = [\quad] \text{ et } \dim(b^{[2]}) = [\quad]$$

$$\text{donc } \dim(S^{[2]}) = \quad = [\quad]$$

La sortie de la deuxième couche cachée $a^{[2]}$ s'écrit alors

$a^{[2]} = \quad \dim(a^{[2]}) = [\quad]$

La sortie pondérée de la troisième couche cachée $a^{[3]}$ qui est la sortie du réseau s'écrit alors

$$S^{[3]} =$$

$$\dim(w^{[3]}) = [\quad] ; \dim(a^{[2]}) = [\quad] \text{ et } \dim(b^{[3]}) = [\quad]$$

$$\text{donc } \dim(S^{[3]}) = \quad = [\quad]$$

La sortie de la troisième couche cachée $a^{[3]}$ qui est la sortie du réseau s'écrit alors

$Y = a^{[3]} = \quad \dim(a^{[3]}) = [\quad]$

4.3.1.3. Exemple de propagation des données dans un PMC

Soit l'architecture d'un réseau de neurone de type PMC décrite par la structure 3-4-1. Chaque neurone est décrit par une fonction d'activation de type sigmoïde.

1. Donner les matrices des paramètres de cette structure.

Utilisant la base des données suivante :

Entrées			Sortie
Age (x_1)	Puissance(x_2)	Kilométrage(x_3)	Prix
60	4	110	22
48	7	90	31
12	4	20	45
30	5	45	43

2. Donner la matrice des entrées sachant que la normalisation se fait respectivement selon les valeurs suivantes : 200, 20, 500, 200.

3. Donner les valeurs de la matrice de sortie du réseau

Ecriture matricielle de la sortie de la première couche cachée.

--

Ecriture matricielle de la sortie de la couche de sortie.

--

Le calcul est comme suit :

--

4. Calculer l'erreur entre les sorties prédites par le réseau et celles de la base.

4.3.2 Les réseaux de neurones convolutifs CNNs

4.3.2.1. Présentation

L'architecture des réseaux de neurone convolutifs est développée par ----- . Cette architecture se base sur l'opérateur de la ----- . Cet opérateur est un opérateur mathématique qui permet d'extraire des caractéristiques importantes des images ou des signaux. Un CNN est composé par ----- : un premier bloc formé ----- et un deuxième bloc formé ----- . Ce type de réseau de neurone (CNN) a montré des performances spectaculaires dans les applications de traitement de la parole et surtout dans les applications de ----- . Beaucoup de CNN ont été développé sous plusieurs versions comme les réseaux **YOLO V_XX**. Ils ont montré des bonnes performances dans divers domaines de **classification** et de **détection des objets**. La figure ci-dessous montre un CNN classifieur à 3 classes avec 3 couches de convolution et un PMC à 3 couches entièrement connectées.

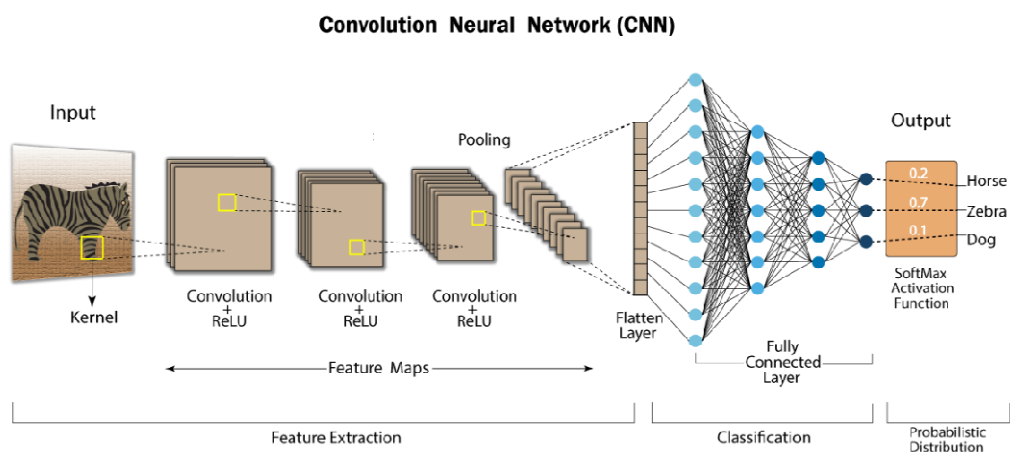


Figure 5.6. Représentation d'un CNN

Remarque : Dans cette figure il existe la notion de pooling, le kernel et la couche aplatit (flatten layer). Ces notions seront expliquées dans la partie qui suit.

3.3.2.2. Notion et principe de convolution

✓ Principe de la convolution

La convolution est le balayage d'un kernel sur une image originale I pour obtenir une nouvelle image I' . Un kernel est un filtre caractérisé par une matrice de dimension ----- . Dans le cas où le filtre est de taille 3x3 on parle alors de ---- coefficients qui le caractérisent.

La convolution d'une image consiste à balayer l'image d'origine par un filtre (ou kernel) pixel par pixel en commençant par la première ligne vers la dernière.

L'image d'origine I

10	20	30	40	50
20	30	40	50	60
30	40	50	60	70
40	50	60	70	80
50	60	70	80	90

Soit le filtre H s'écrit :

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

L'image traitée $I' = I * H$ alors pour un pixel $I'(1,1) = \text{-----}$

Le calcul se fait comme suit : $I'(1,1) =$

Ainsi on peut construire la nouvelle image I' de dimension [3x3]

Ce résultat ne traite pas les pixels de bords. Ce qui donne une image I' de dimension inférieur à l'image d'origine. Ceci peut être utilisé si on veut pour diminuer petit à petit la dimension de l'image.

Ainsi, la dimension de l'image traitée I' suit la relation suivante :

$$i' = (i - k) + 1$$

avec k est la dimension du filtre et i le nombre des lignes ou le nombre des colonnes i_L ou i_W .

- **Traitement des pixels de bord et conservation de dimension : Notion de -----**

Dans le cas où on veut **garder** ----- de l'image d'origine, on doit traiter les pixels de bords (C'est la notion de '-----'). En effet, on ajoute (ou on fait un remplissage) de ----- **lignes et colonnes** aux bords de l'image. Dans ce cas il existe 2 principales méthodes :

- ✓ Soit en ajoutant des pixels virtuels au bord de l'image qui ont la valeur 0.

0	0	0	0	0	0	0
0	10	20	30	40	50	0
0	20	30	40	50	60	0
0	30	40	50	60	70	0
0	40	50	60	70	80	0
0	50	60	70	80	90	0
0	0	0	0	0	0	0

- ✓ Soit en ajoutant des pixels virtuels qui ont les mêmes valeurs que les pixels de bords ou des valeurs symétriques par rapport à la ligne ou la colonne de bord.

30	20	30	40	50	60	50
20	10	20	30	40	50	40
30	20	30	40	50	60	50
40	30	40	50	60	70	60
50	40	50	60	70	80	70
60	50	60	70	80	90	80
50	40	50	60	70	80	70

Remarque :

Le résultat de l'application de la convolution dans les cas présentés **n'est pas le même ce qui affecte le traitement des résultats et introduit du bruit dans le traitement.**

• Pas de convolution (-----)

Dans certains cas on peut adapter la notion du pas de convolution (qu'on appelle -----). Cette notion définie si le balayage se fait **pixel par pixel** ou **pixel par un saut d'un pixel** ou par un saut de 's' pixels. L'introduction de la notion du pas de convolution permet de -----
-----et par -----.

Cas du pas = 2

10	20	30	40	50
20	30	40	50	60
30	40	50	60	70
40	50	60	70	80
50	60	70	80	90

Le filtre H s'écrit :

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

L'image traitée $I' = I * H$ est :

Tenant compte du padding noté par : 'p' et le pas de convolution (stride) noté par : 's', l'expression de la dimension de l'image traitée (carte caractéristique ou features map) devient :

$$i' =$$

Il faut tenir compte que sans remplissage $p=0$ (No padding $p=0$) avec remplissage $p=1$.

3.3.2.3. Couche de convolution

Dans le cas d'un CNN, une couche de convolution consiste à appliquer plusieurs filtres à une image.

Le nombre de filtre définit la notion de la profondeur de la couche notée par n_f .

Dans le cas d'un seul filtre (profondeur $n_f=1$) la représentation est comme suit :

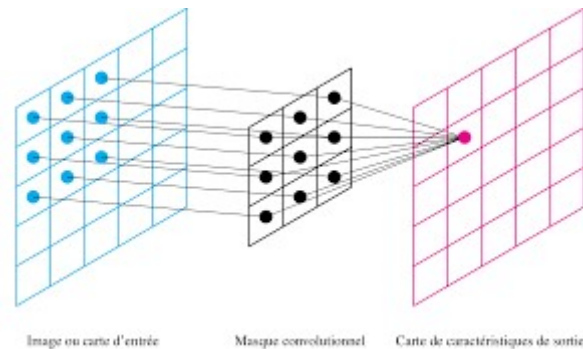


Figure 5.7. Représentation d'une couche de convolution avec un seul filtre.

Pour calculer le premier pixel il faut faire la somme pondérée des pixels de l'image avec les paramètres du filtre ce qui nous rappelle la forme mathématique d'un neurone artificiel. Dans le cas d'un filtre 3x3 on a 9 paramètres w_{ij} (paramètres du filtre) avec 9 entrées (pixels) l'équation s'écrit :

$$s_j =$$

D'une façon générale dans les couches cachées on note la sortie des neurones par a_j et non pas y_j .

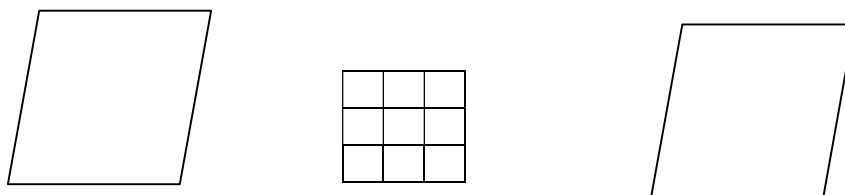
Dans ce cas la fonction d'activation est de la forme ----- qui s'écrit :

$$a_j = \begin{cases} s_j & \text{si } s_j \geq 0 \\ 0 & \text{si } s_j < 0 \end{cases}$$

Cette fonction permet d'annuler les valeurs négatives et de garder les valeurs positives.

On peut conclure que le premier pixel est le résultat de l'activation d'un neurone. Et par suite si on a une image de dimension **10x10** traitée par un filtre **3x3** on aura (-----) paramètres.

- Dans le cas d'un seul filtre la représentation est comme suit :



NB : il faut savoir que le **nombre des paramètres est indépendant** ----- mais il dépend de la **taille** ----- . Dans le cas ou on une image 1980x1080 le nombre de paramètres reste le même. En contre partie le ----- **augmente** considérablement.

- Dans le cas de 4 filtres 7x7 la représentation devient :

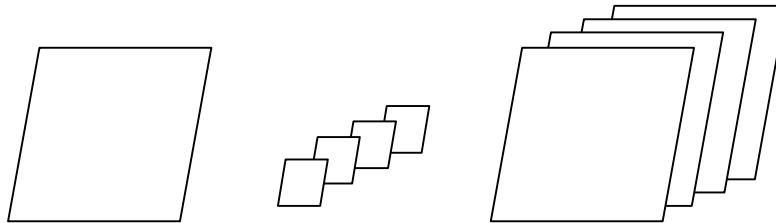


Figure 5.8. Représentation d'une couche de convolution avec 4 filtres.

Tenant compte que pour chaque filtre possède un biais, le nombre de paramètres dans la couche de convolution est : -----

- Dans le cas générale on représente la couche de convolution par un cube de profondeur égale au **nombre de filtre** dans cette couche. On définit alors l'hyper paramètre : qui est la profondeur dans la couche cachée n_f (qui 'est le nombre de filtre dans la couche). La figure ci-dessous représente un bloc de convolution à deux couches de convolution de profondeur respectives $n_{f1}=16$ et $n_{f2}=32$ de dimension 5x5.

Figure 5.9. Représentation de deux couches de convolution avec 16 et 32 filtres.

Dans ce cas le nombre des paramètres selon les couches est :

Nombre de paramètres dans la 1^{ère} couche de convolution : -----

Nombre de paramètres dans la 2^{ème} couche de convolution : -----

3.3.2.4. Notion de pooling

La notion de pooling est appliquée après la couche ----- . Généralement, on intègre l'appellation de cette étape sous le nom de la **couche de pooling**. Cette couche est utilisée pour ----- des cartes caractéristiques tout en conservant l'information importante. Cette réduction permet de ----- d'une part et d'autre part, minimiser **le nombre de paramètres dans la couche du PMC dans le deuxième bloc**.

Le pooling consiste à appliquer des opérateurs mathématiques sur des petites fenêtres de l'image traitée (feature map) qu'on appelle fenêtre de pooling. Ces opérateurs sont généralement : le max ou la moyenne. Les principales méthodes de pooling sont :

- **Maxpooling** : sélectionne la valeur maximale dans chaque fenêtre.

Exple : maxpooling 2x2

10	15	20	25
15	20	25	30
20	25	30	35
25	30	35	40

On aura comme résultat

- **Averagepooling** : calcule la moyenne des valeurs dans chaque fenêtre

Exple : averagepooling 2x2

10	15	20	25
15	20	25	30
20	25	30	35
25	30	35	40

On aura comme résultat

D'après ces exemples, on peut conclure qu'il existe deux hyper paramètres de cette couche : la dimension ----- et le ----- . Dans les deux exemples précédents la dimension est ----- et le pas s=-----.

La dimension de la nouvelle carte caractéristique s'écrit :

$i' =$	(car toujours $p = 0$ dans le pooling)
--------	--

Exemple

Application de couche de pooling 3x3 avec un pas de 1 :

10	20	30	40	50
20	30	40	50	60
30	40	50	60	70
40	50	60	70	80
50	60	70	80	90

Dans la couche de pooling il n'existe aucun ----- à ajuster.

Le type, la taille et le pas de pooling ----- dans les CNN.

10	15	20	25
15	20	25	30
20	25	30	35
25	30	35	40

[illegible]

✓ _____,

✓ _____,

✓ _____,

✓ _____

[illegible]

Exemple :

3x3 conv 64_P1_S1

On 64 filtres de tailles 3x3 tenant compte du padding avec un pas de convolution égale à 1.

Le nombre de paramètres : $(3 \times 3 + 1) \times 64$

Dans le cas où le pas = 2 le nombre des paramètres reste le même mais le temps de calcul est divisé par 2.

Dans le cas où on a plusieurs couches cachées on note l'indice de la couche en exposant de chaque caractéristique.

Exemple :

✓ -----,
✓ -----,
✓ -----,
✓ -----.

La dimension de la sortie s'écrit :

$i^{[l]} =$

On peut noter aussi :

Dimension de longueur de i : $i_L^{[l]} =$

Dimension de longueur de i : $i_w^{[l]} =$

- Couche de pooling

C'est la couche qui permet de garder les principales caractéristiques de la carte des caractéristiques tout en réduisant leurs dimensions. Ce type de couche est caractérisé par le nombre du paramètre égal à 0. Ainsi la dimension de la carte caractéristique de sortie s'écrit :

$i' =$

- Couche entièrement connectée

Ce type de couche constitue la première étape de la classification de l'image d'entrée. Dans un CNN on peut avoir une ou plusieurs couches cachées entièrement connectées. Dans ce cas, le calcul des paramètres de ce type de couche se fait comme dans le cas d'un PMC. Toutefois il faut tenir en compte que généralement les fonctions de transfert des couches cachées sont des sigmoïdes.

3.3.2.7. Exemple d'architecture d'un CNN : Le NET-5

Soit l'architecture d'un CNN qui traite une image d'entrée de dimension 32x32. L'architecture est décrite comme suit :

- Première couche :
 - ✓ Première couche de convolution conv1 : (k=5,S=1,nf=6)
 - ✓ Première couche de pooling Pool1 : (F=2,S=2)
 - Deuxième couche :
 - ✓ Deuxième couche de convolution conv2 : (k=5,S=1,nf=6)
 - ✓ Deuxième couche de pooling poll2 : (F=2,S=2)
 - Couche entièrement connectées : 120-84-10
1. Donner une représentation de l'architecture du CNN.
 2. Donner le nombre de paramètres par couche.

